

An AI-Native Event-Driven Edge–Cloud Architecture for Autonomous Network Security in Campus Infrastructure

Hoang Tu Trinh and Tien Thanh Cao

Faculty of Information Technology

Ho Chi Minh City University of Foreign Languages – Information Technology

Ho Chi Minh City, Vietnam

Email: 23dh113972@st.huflit.edu.vn, thanhct@huflit.edu.vn

I. INTRODUCTION

Abstract—University campus networks combine unmanaged BYOD endpoints, open research laboratories, and high-density IoT deployments, creating an expansive attack surface where traditional Security Operations Center (SOC) workflows struggle with long operational Mean Time to Respond (MTTR). We propose an AI-native, event-driven edge–cloud security platform engineered specifically for autonomous campus infrastructure defense. The architecture deploys lightweight INT8-quantized TCN–GRU sequence models at network edge gateways to score non-DPI flow telemetry. Enriched security events are carried asynchronously over a Redis Streams consumer group to a cloud risk resolver and Deep Q-Network (DQN) Software-Defined Networking (SDN) policy that emits SOAR playbooks; installing them in the dataplane is outside what we measure. Detection is evaluated on 20,000 stratified real InSDN windows [1]. Separately, a 1,000-trial control-path microbenchmark on one prepared held-out window (INT8 inference, HMAC `principal_id`, Redis `XADD/XREADGROUP`, DQN, playbook construction; no dataplane) achieves a 4.793 ms median and a 7.202 ms p99 on localhost Redis 7.2.4. Because stride-1 windows overlap by nine of ten timesteps, we evaluate under a blocked and purged split that removes stride-1 index overlap, with a train-only scaler; comparing that protocol to a conventional random split yields a near-null leakage effect ($|\Delta F1| < 0.002$ at fixed τ). Under the controlled protocol the INT8 detector reaches $F1 = 0.9340$ at $\tau = 0.65$ with a 0.048 MB footprint and negligible quantization loss ($|\Delta F1| \leq 0.0002$). We further report five negative results: gradient-boosted trees on the same features outperform the proposed detector (LightGBM $F1 = 0.9886$ against 0.9723), leaving it only a footprint advantage; reconstruction-error scoring is unusable on this attack-heavy corpus ($F1 \approx 0.035$), so detection rests entirely on the supervised head; our edge CPU budget is missed (36.0% of one logical CPU at a sustained 10,000 windows/s against a pre-specified 10%, with `torch.set_num_threads(1)`); zero-shot transfer to CIC-IDS2017 collapses ($F1 = 0.456$); and localhost Redis Streams is not faster than a tight REST loopback, so the bus is justified by the multi-consumer deployment model rather than single-host IPC speed. We provide an ablation study, policy benchmarks against a brute-force oracle, and a seeded reproducibility artifact.

Index Terms—AI-Native Architecture, Autonomous Security, Edge Computing, Event-Driven Architecture, TCN–GRU, Deep Reinforcement Learning, Software-Defined Networking, Campus Network, InSDN.

Artificial Intelligence (AI) is reshaping higher-education digital infrastructure, but university campus cybersecurity faces an increasingly intractable threat landscape: IoT sensors, student Bring-Your-Own-Device (BYOD) endpoints, open research laboratories, and administrative databases share one network fabric. Centralized SIEM and human-in-the-loop SOC pipelines exhibit long MTTR during volumetric DDoS, ransomware lateral movement, or data exfiltration [4], causing analyst burnout and delayed containment.

To bridge this gap we propose an **AI-Native Event-Driven Edge–Cloud Security Architecture**. Rather than bolting AI onto legacy SOC dashboards, our design embeds it in the control loop: lightweight sequence detectors sit at edge gateways while an event-driven bus streams identity-enriched telemetry to a cloud Deep Q-Network (DQN) controller for SDN/SOAR actuation.

A. Research Questions

This investigation is guided by four formal Research Questions (RQs):

- **RQ1 (Control-Path Latency):** Can an event-driven edge–cloud architecture reduce the software decision path, from a prepared edge telemetry window entering inference to containment-playbook emission, to sub-10 ms levels?
- **RQ2 (Quantization & Accuracy):** Does INT8 dynamic quantization maintain NIDS classification accuracy while satisfying an edge gateway *model-footprint* constraint (≤ 1 MB of model storage)?
- **RQ3 (Reinforcement Learning Efficiency):** How closely can a DQN approximate the reward-optimal action under a modeled single-step contention environment relative to static and heuristic policies?
- **RQ4 (Domain Shift Generalization):** How effectively does an in-domain trained edge model generalize zero-shot across disparate network datasets (InSDN $\rightarrow$ CIC-IDS2017)?

B. Formal Hypotheses

We test two explicit hypotheses:

- **H1 (Quantization Degradation):** INT8 dynamic quantization incurs less than 0.5% $F1$ -score degradation ($|\Delta F1| \leq 0.005$) compared to full-precision FP32 models.
- **H2 (Edge Resource Feasibility):** Edge inference requires $\leq 10\%$ of one logical CPU (psutil convention: 100% = one core fully busy) under a sustained offered load of 10,000 telemetry windows/sec with a single PyTorch host thread.

C. Contributions

The principal contributions of this work are:

- 1) **Decoupled AI-Native Pipeline:** An event-driven architecture combining PyTorch INT8 TCN-GRU edge detectors, simulated identity-context fusion, HMAC principal pseudonymization, and a Redis Streams consumer-group event transport, with a DQN policy that *emits* SDN/SOAR action descriptors for forwarding, rate limiting, rerouting, and isolation. We measure the software path that produces those descriptors, not their installation in the dataplane.
- 2) **Empirical Validation & Negative Results:** Microbenchmarks on real InSDN [1] traffic, with Redis Streams in the timed path, ablation, policy benchmarks against a brute-force oracle, saturation analysis, and a CIC-IDS2017 [2] holdout. We report five negative results — failure of reconstruction-error scoring, a near-null leakage effect from stride-1 overlap, rejection of our edge CPU budget (H2), severe zero-shot transfer failure, and gradient-boosted trees outperforming the proposed detector — that bound where this design is deployable.
- 3) **Seeded Reproducibility Artifact:** A reproducibility artifact regenerating every quantitative result table in Section VI from one seeded entry point, at <https://github.com/thtsec/icai-fai2026>.

II. RELATED WORK

SDN intrusion datasets such as InSDN [1] and CIC-IDS2017 [2] have established standard benchmarks for network intrusion detection. Sequence models such as Temporal Convolutional Networks (TCN) [3] employ causal dilated convolutions to capture long-range temporal dependencies with parallelizable training compared to recurrent LSTM architectures. Deep Reinforcement Learning (DRL) has been applied to DDoS defense and 6G control loops [4]. Closing the loop is not itself novel: Liu *et al.* [24] mitigate SDN flooding online with DRL, and Yungaicela-Naula *et al.* [25] integrate a DL detector with a DRL mitigator on Mininet and ONOS and evaluate autonomous response against live traffic. We therefore make no priority claim over autonomous mitigation. What is less well covered is the composition: an event-driven edge-cloud split in which quantized detectors run on the gateway and policy runs in the cloud, with the software path between them quantified end to end, and with leakage

control, quantization cost, domain shift and negative results reported rather than elided.

Compressing detectors for constrained gateways is an active line: pruned and quantized LSTM ensembles on Raspberry Pi [17] and INT8 TensorFlow Lite CNNs on ARM Cortex-M [18] both find INT8 accuracy-neutral at far lower footprint, consistent with our H1, but both stop at the detector and close no loop onto actuation. Conversely, surveys of agent-driven security operations [19] catalogue triage automation while noting that end-to-end autonomous remediation remains prospective; our work sits between the two. A parallel thread concerns evaluation rigour: Arp *et al.* [20] show data snooping is endemic in learning-based security papers and inflates results, and leakage-free reassessments of ML-NIDS confirm preprocessing alone moves headline metrics [21]. We adopt those recommendations in Section V-B and report their cost in Section VI-H.

Table I positions our architecture against this literature by scope, restricted to properties each work claims for itself. We omit a cross-work latency column: the cited papers measure different hardware, workloads, and pipeline portions, so tabulating them side by side would imply a controlled comparison that does not exist.

III. THREAT MODEL & CAMPUS ATTACK VECTORS

A. Adversary Capabilities and Attack Vectors

We consider an active adversary $\mathcal{A}$, either external or an internal compromised BYOD/IoT endpoint with campus subnet access, capable of volumetric DoS, ARP/MAC spoofing, lateral reconnaissance, and credential exfiltration. $\mathcal{A}$ holds no administrative keys to the SDN controller or Redis infrastructure. We do not model an adaptive adversary who crafts inputs against the detector itself; that omission is revisited in Section VII-A.

B. Trust Boundaries and Attack Surface

Fig. 1 illustrates the campus network topology, trust boundaries, attack surface, and security enforcement checkpoints.

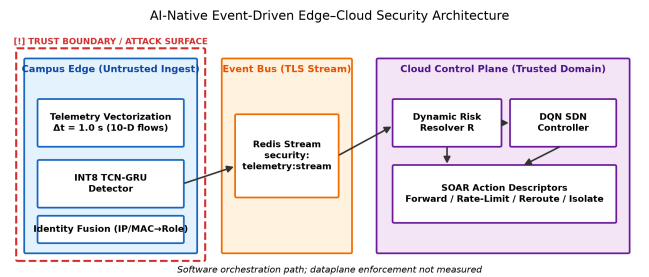


Fig. 1. AI-native event-driven edge-cloud software orchestration path and trust boundary. SOAR boxes are action descriptors only; dataplane enforcement is not measured.

IV. AI-NATIVE SYSTEM ARCHITECTURE

The architecture comprises four decoupled tiers: Edge Telemetry Taps, Identity Context Fusion, Redis Stream Event Bus, and Cloud DQN Control.

TABLE I

SCOPE COMPARISON WITH RELATED LITERATURE. ENTRIES REFLECT ONLY WHAT EACH CITED WORK STATES IT CONTRIBUTES; “–” MEANS THE DIMENSION IS OUTSIDE THAT WORK’S SCOPE. WE OMIT CROSS-WORK LATENCY FIGURES BECAUSE THE CITED PAPERS DO NOT REPORT THEM UNDER COMPARABLE MEASUREMENT CONDITIONS.

Work	Contribution Type	Edge AI	INT8 Quantized	RL Control	Dataplane Enforcement
Sharafaldin <i>et al.</i> (2018) [2]	Benchmark dataset	–	–	–	–
Bai <i>et al.</i> (2018) [3]	Sequence architecture (TCN)	–	–	–	–
Tang <i>et al.</i> (2018) [13]	SDN intrusion detection (DNN)	No	No	No	Detection only
Roopak <i>et al.</i> (2019) [14]	DL models for IoT security	No	No	No	Detection only
Zhang <i>et al.</i> (2019) [12]	Survey	–	–	–	–
Elsayed <i>et al.</i> (2020) [1]	Benchmark dataset (InSDN)	–	–	–	–
Nguyen <i>et al.</i> (2023) [15]	SOAR survey + research agenda	No	No	Proposed	Proposed, not evaluated
Kianpisheh & Taleb (2024) [4]	Federated DRL for DDoS	Federated	No	DQN [5]	Simulation only
Musthafa <i>et al.</i> (2025) [17]	Compressed ensemble IDS on Raspberry Pi	Yes	Yes (post-training)	No	Detection only
Ullah <i>et al.</i> (2026) [18]	Quantized IDS for constrained IoT	Yes	Yes (TFLite INT8)	No	Detection only
Ours	Integrated pipeline	TCN-GRU	Yes (0.048 MB)	DQN	No (action descriptors only)

TABLE II

STRIDE & MITRE ATT&CK CAMPUS THREAT MATRIX.

THREAT-TO-MITIGATION MAPPINGS *considered by the architecture*;
 REPUTATION IS OUTSIDE SCOPE (NO TAMPER-EVIDENT AUDIT TRAIL).
 PROTOTYPE PLAYBOOKS IMPLEMENT FORWARD / RATE-LIMIT / REROUTE /
 ISOLATE DESCRIPTORS ONLY.

Category	Target	MITRE ID	Mitigation Strategy
Spoofing	Rogue Wi-Fi AP / MAC	T1557	Identity fusion MAC/IP verification
Tampering	IoT telemetry data	T1565	Edge TCN-GRU sequence scoring
Information Disclosure	Research DB server	T1048	Role-aware byte-rate capping
Denial of Service	Campus SDN controller	T1498	Rate-limit / isolate (<i>flow_mod</i> drop)
Elevation of Privilege	Campus LDAP directory	T1078	Dynamic risk escalation

A. Edge Telemetry and TCN-GRU Detector

Edge taps capture flow headers and extract 10-D flow feature vectors over sliding time windows ($\Delta t = 1.0$ s). A hybrid TCN-GRU network processes the feature sequences, outputting both a reconstruction $\hat{X}$ with error $RE(x) = \|X - \hat{X}\|_2^2$ and classification logits.

Training objective. The model is trained *multi-task*, which determines how the results should be read. Each epoch runs a reconstruction pass minimizing $MSE(X, \hat{X})$ over *normal windows only*, then a joint pass minimizing $MSE(X, \hat{X}) + \lambda CE(\text{logits}, y)$ over all windows with $\lambda = 0.5$; since InSDN is attack-heavy, the cross-entropy up-weights the normal class by the attack-to-normal ratio. The detector is therefore *not* unsupervised: it requires labels, and the autoencoder branch is an auxiliary representation objective, not the decision rule. Detection evaluates the supervised head, $P(\text{attack}) = 1 - P(\text{normal})$, against τ ; reconstruction error is not the operational score, for the reason in Section VI-C.

B. Dynamic INT8 Quantization

To satisfy memory-constrained edge gateways, PyTorch [11] dynamic INT8 quantization compresses linear and GRU layer weight matrices from 32-bit floating point to 8-bit integers. We report no feature attribution study, since our artifact does not implement one and we prefer to leave the question open rather than assert an unmeasured importance ranking. Identifying which of the 10 features carry the discriminative signal, and

whether that ranking is what fails to transfer (Section VI-I), is future work.

C. Identity Context Fusion & Privacy Preservation

Edge security events are enriched against a *simulated* local DHCP/RADIUS lease table (hard-coded prototype map; the artifact does not query live directory services) to append role/trust context:

$$\mathcal{E} = \{\text{principal_id}, P(\text{attack}), \text{UserRole}, \text{TrustScore}\}. \quad (1)$$

Cleartext IP/MAC remain edge-local. Before Redis transport the edge emits $\text{principal_id} = \text{hex}(\text{HMAC-SHA256}(k, \text{src_ip}))$ with deployment secret k (environment variable `ICAI_PSEUDONYM_SECRET`, else a fixed development key for reproducible benches). Unkeyed SHA-256 is *not* used. Raw payload inspection (DPI) is omitted entirely.

D. Cloud Risk Engine and DQN SDN Control

Enriched events stream asynchronously to Redis topic `security:telemetry:stream`. The cloud risk engine calculates a composite risk index R over the detector’s attack probability, the identity trust score, and asset criticality:

$$R = w_1 P(\text{attack}) + w_2 (1 - \text{TrustScore}) + w_3 \text{AssetCriticality}, \quad (2)$$

with $(w_1, w_2, w_3) = (0.60, 0.25, 0.15)$ in all reported experiments; the fused term is the classifier probability, not $RE(x)$. When identity is ablated the trust term is undefined and its weight folds into detection, giving $R = 0.85 P(\text{attack}) + 0.15 \text{AssetCriticality}$, keeping scores comparable across arms. Severity bands map R to preferred containment intensity, but the timed prototype constructs descriptors only for the four DQN actions below.

Two caveats. The weights are hand-set, not learned, and encode a design prior we do not claim is optimal. More importantly, InSDN carries no asset inventory, so `AssetCriticality` is drawn i.i.d. uniform on $[0, 1]$ in every experiment and is pure noise; the third term therefore *degrades* our numbers, and the risk-engine benefit reported later comes from the trust term alone.

At the selected severity, the Deep Q-Network (DQN) policy agent [5] evaluates state vector s_t and selects containment action $a_t \in \{a_0 : \text{Forward}, a_1 : \text{Rate-Limit}, a_2 : \text{Reroute}, a_3 : \text{Isolate}\}$; the latency harness constructs the matching SOAR action descriptor for that a_t [15]. Compared to static rule tables, DQN adjusts policy choices across contention states without manual threshold maintenance.

V. EXPERIMENTAL METHODOLOGY

A. Hardware & Software Testbed Specification

Quantitative results in Section VI were measured on the host in Table III. Mininet / OVS / Ryu [9], [10], [16] are cited as the intended campus dataplane sandbox for future closed-loop installs; they are *not* present as an executable topology in this artifact and are *not* on the timed path for the reported tables. Those tables exercise the detector, simulated identity context, HMAC principalization, Redis Streams, DQN policy, and playbook-descriptor construction on localhost Redis 7.2.4 (Docker), excluding `flow_mod` installation.

TABLE III
MEASUREMENT HOST FOR HEADLINE TABLES (SECTION VI)

Parameter	Specification
CPU	13th Gen Intel Core i7-13620H (10 cores / 16 threads)
RAM	16 GB
Operating System	Windows 11 (build 22H2), native Python 3.11.9
Python / PyTorch	PyTorch 2.13.0+cpu, scikit-learn (artifact venv)
Event Bus	Redis 7.2.4 (Docker, localhost:6379, Streams)
Dataset archives	Frozen InSDN / CIC window NPZs (see artifact datasets/)
Timer Precision	Python <code>time.perf_counter_ns()</code>

B. Dataset Sampling & Leakage Control

We evaluate on frozen window archives derived from public InSDN [1] (and CIC-IDS2017 [2] for transfer). From the public InSDN CSV(s) ($\approx 343,939$ flows) we took a label-stratified subsample of 50,000 rows (seed 42), *shuffled* that subsample with the same seed, then formed contiguous windows ($T = 10$, stride 1) yielding 49,991 windows (40,043 attack / 9,948 normal). Each window label is the label of its *last* flow — not an any-attack-in-window rule — so the $\approx 80\%$ attack rate matches the stratified flow mix. Because shuffling precedes windowing, these are length- T contexts over an index order, not guaranteed capture-time trajectories. The artifact ships the NPZs with SHA-256 manifests and a `datasets/build_windows.py` recipe; raw vendor CSVs are not redistributed.

Window i shares $T - 1 = 9$ of its 10 timesteps with each neighbour, so an i.i.d. random split places near-duplicate observations on both sides of the train/test boundary — precisely the sampling and snooping pitfalls catalogued in [20], [21]. Our protocol instead (i) cuts the index-ordered window corpus into 20 contiguous blocks and assigns whole blocks to train or test; (ii) *purges* 9 windows from both ends of every block, guaranteeing no surviving training window shares even a single timestep with a test window; (iii) stratified-subsamples each side independently to 14,000 train and 6,000

test windows, preserving class balance; and (iv) fits the scaler on the training partition only, transforming the test partition and the CIC-IDS2017 holdout with those frozen statistics. This controls stride-1 *index-overlap* leakage; it is not wall-clock session isolation over released CSV order. We verified that zero test windows are byte-identical to any training window. Section VI-H quantifies what this protocol is actually worth.

VI. QUANTITATIVE EVALUATION & RESULTS

Reading the $F1$ values. Each figure belongs to a distinct protocol and they are not interchangeable. 0.9340 is the detector at the fixed $\tau = 0.65$, the deployable accuracy. 0.9679 is the same detector at $\tau^* = 0.0672$ *selected on the test split*, an oracle bound. 0.9723 is the corresponding INT8 oracle figure in Section VI-E, differing because that harness trains baselines first and advances the RNG — which bounds how finely any figure should be read. 0.9616 is the full pipeline with τ tuned on the *training* split (Section VI-F). Compare only within an experiment.

All results below use the leakage-controlled protocol of Section V-B.

A. Latency Breakdown (RQ1)

Scope. We time the *software control path*: edge INT8 inference on one real InSDN prepared window ($X_{te}[:1]$ from the blocked test partition) using the trained detector checkpoint quantized to INT8, simulated identity enrichment with HMAC-SHA256 `principal_id`, Redis Streams `XADD` followed by consumer-group `XREADGROUP/XACK` on live Redis 7.2.4, DQN action selection, and SOAR playbook *construction* for the *selected* DQN action. It excludes packet capture, feature extraction, windowing, controller transmission, and `flow_mod` installation. Redis runs on localhost; cross-host RTT is not included. The latency harness uses the DQN *architecture* for forward-path timing; Table IX separately evaluates the trained policy.

Table IV presents the per-stage breakdown across 1,000 trials, timed with `time.perf_counter_ns()`, after 50 warm-up iterations. **We report percentiles rather than mean $\pm$ SD** because the per-trial distribution is heavy-tailed. HMAC identity enrichment is visible but small (median 0.030 ms). Median control-path latency is 4.793 ms, with edge inference at 60.0% and Redis Streams at 32.0% of the median total; the p99 is 7.202 ms. The sub-10 ms target of **RQ1** is therefore met for this Redis-inclusive software path at the 99th percentile.

Reconciling detector latencies. Per-inference figures for the same INT8 model differ across harnesses (pipeline loop vs. tight comparison loop) under different scaffolding rather than in contradiction. Per Section VII-A, absolute timings on this host should be read to roughly a factor of two.

B. Detection Performance & Threshold Sensitivity (RQ2, H1)

Table V shows metric variation across decision threshold τ under the leakage-controlled protocol. The detector is strongly precision-biased in this regime: at the fixed operating point

TABLE IV

PER-STAGE SOFTWARE CONTROL-PATH LATENCY ($n = 1000$ TRIALS, REDIS 7.2.4 LOCALHOST, HMAC PRINCIPALS). EXCLUDES DATAPLANE ENFORCEMENT. SHARES ARE OF THE MEDIAN TOTAL AND DO NOT SUM TO 100 BECAUSE MEDIAN ARE NOT ADDITIVE.

Stage	Median (ms)	p95 (ms)	p99 (ms)	%
Edge TCN-GRU Inference	2.875	4.327	4.797	60.0
Identity Context Fusion (HMAC)	0.030	0.046	0.069	0.6
Redis Streams (XADD+XREADGROUP)	1.534	2.236	2.629	32.0
Cloud DQN Policy	0.188	0.285	0.374	3.9
SOAR Playbook Construction	0.003	0.005	0.007	0.1
Control-Path Total	4.793	6.604	7.202	100.0

$\tau = 0.65$ it yields $F1 = 0.9340$ (Precision = 0.978, Recall = 0.894, FPR = 8.20%), and over the reported grid $F1$ peaks at 0.955 at $\tau = 0.35$. Searching τ freely over the test split reaches $F1 = 0.9679$ at $\tau^* = 0.0672$, the operating point Section VI-E uses for model comparison. We flag the protocol because it matters: a test-selected threshold is an oracle — the test-tuning pitfall of [20] — so 0.9679 bounds what the detector could achieve, not what it would.

TABLE V

DETECTION METRICS VS. DECISION THRESHOLD τ ON REAL INSDN

Threshold τ	Precision	Recall	F1-Score	FPR (%)	FNR (%)
0.35	0.969	0.942	0.955	12.13	5.8
0.50	0.974	0.926	0.949	10.13	7.4
0.65	0.978	0.894	0.934	8.20	10.6
0.75	0.980	0.869	0.921	7.03	13.1
0.85	0.988	0.799	0.883	3.93	20.1

C. Why Reconstruction Error Is Not the Decision Rule

Since much of the sequence-autoencoder IDS literature scores on reconstruction error alone, it is worth reporting which of our two outputs actually works. We evaluated the standard unsupervised rule $\tau = \mu + k\sigma$, with μ, σ estimated on reconstruction errors of *normal training* windows, for $k \in \{0.5, \dots, 3.0\}$. It fails comprehensively: the best setting $k = 0.5$ yields $F1 = 0.0354$; detection performance is thus entirely attributable to the supervised head, and the architecture should be cited as a supervised sequence classifier with an auxiliary reconstruction objective. The classifier head defaults to six logits for checkpoint compatibility, but training collapses labels to binary normal-vs-attack and scores $1 - P(\text{normal})$, so classes 2–5 are unsupervised in the reported experiments.

D. Throughput Saturation & Edge Feasibility (H2)

Per-inference latency and throughput reconcile only once the batching regime is stated, so we report both, each as the median of 5 repeats. Run flat out with a single PyTorch host thread, the detector saturates at 662 windows/s unbatched, rising through 10,302 at batch = 32 to 51,239 at batch = 512: batching, not the model, is what reaches campus scale. Saturation remains the least stable quantity we measure, so we rest no claim on it. Table VI instead reports the *paced* offered load at batch = 128.

CPU

metric.

We

report

`psutil.Process.cpu_percent()` over the paced wall-clock window with *no* division by `cpu_count()`: 100% means one logical CPU fully busy. Dividing by logical CPU count would convert the reading into a fraction of total machine capacity and must not be labelled “% of one core.” PyTorch is pinned with `torch.set_num_threads(1)` so the unit matches H2. At low offered rates the window is sleep-dominated, so the reading understates busy-period cost; the H2 decision uses the 10,000 windows/s row.

H2 is rejected. The hypothesis asserted $\leq 10\%$ of one logical CPU at a sustained 10,000 windows/s. The system sustains that load (9,998 of 10,000 offered windows/s in every repeat) while consuming a median 36.0% of one logical CPU (range 35.0–49.1% across repeats), about $3.6\times$ the budget. Memory is favourable: peak working set is flat at ≈ 541 MB, dominated by the Python/PyTorch runtime rather than the 0.048 MB model. The system is memory- but not CPU-feasible on one commodity logical CPU under this runtime; a compiled ONNX or TensorRT path is the obvious next step.

We deliberately omit a legacy inline-DPI comparison. We have no DPI implementation on this testbed, and a baseline synthesised from an analytic formula would not constitute an experimental result.

TABLE VI

EDGE COST AT PACED OFFERED LOAD (BATCH = 128, `TORCH.SET_NUM_THREADS(1)`), MEDIAN OF 5 REPEATS. CPU IS % OF ONE LOGICAL CPU (100% = ONE CORE BUSY) OVER THE PACED WALL-CLOCK WINDOW, INCLUDING RATE-LIMIT SLEEPS. **LOAD MET** MEANS EVERY REPEAT ACHIEVED AT LEAST 95% OF THE OFFERED THROUGHPUT TARGET; IT IS NOT THE H2 CPU-BUDGET DECISION.

Offered (win/s)	Achieved (win/s)	CPU (%/1 CPU)	Peak WS (MB)	Load Met
100	100.0	0.0	540.6	yes
1,000	999.8	4.3	540.6	yes
5,000	4,999.5	18.8	540.6	yes
10,000	9,998.1	36.0	540.6	yes

E. Detector Baseline Comparison & Quantization Degradation (H1)

Table VII compares our INT8 TCN-GRU against unsupervised, tabular and sequence baselines trained on the identical InSDN split; Transformer AE is a 2-layer encoder-decoder with 4 attention heads and masked reconstruction loss. Every model is scored at its own optimal threshold, since a threshold favouring one architecture need not suit another. *That threshold is selected by searching the test split*, so each $F1$ here is an optimistic bound rather than a deployable operating point; the identical search is applied to every model, so comparison between rows stays internally fair. The tree ensembles receive the window flattened to $T \times d = 100$ features — the same raw information the sequence models see — so the comparison does not handicap them.

Gradient-boosted trees beat the proposed detector, and this is a negative result. LightGBM [22] reaches $F1 = 0.9886$ and XGBoost [23] 0.9880 against 0.9723 for INT8

TCN-GRU, and XGBoost does it at 0.399 ms per inference against 1.935 ms. Random Forest also exceeds TCN-GRU (0.9843). On these features and shuffled index-order contexts, trees outperform TCN-GRU, so this run does not establish a temporal-modeling advantage; capture-order or session-grouped evaluation is required before concluding about temporal signal. One advantage survives: the INT8 model footprint is 0.048 MB against serialized tree objects of 0.845–20.6 MB, but the RQ2 budget was 1 MB and XGBoost fits inside it.

The Isolation Forest row uses `contamination="auto"` on time-averaged windows; on an 80%-attack corpus this heuristic is badly mismatched, so $F1 = 0.2000$ evidences that an untuned classical outlier detector fails at this class balance rather than serving as a tuned baseline.

INT8 quantization reduces the model footprint from 0.104 MB to 0.048 MB at negligible accuracy cost ($|\Delta F1| = 0.0002$), supporting hypothesis **H1**. Since the RQ2 budget was 1 MB and the FP32 model already meets it at 0.104 MB, the honest reading of H1 is that quantization is *harmless* rather than necessary for gateways at this memory scale.

TABLE VII

BASLINE & MODEL QUANTIZATION COMPARISON ON REAL INSDN. EACH MODEL IS SCORED AT ITS OWN $F1$ -OPTIMAL THRESHOLD SELECTED ON THE TEST SPLIT (SEE TEXT). ITALICS MARK MODELS THAT OUTPERFORM THE PROPOSED DETECTOR. SIZE IS THE PARAMETER FOOTPRINT FOR PYTORCH MODELS ONLY.

Model Algorithm	F1-Score	Latency (ms)	Size (MB)
Isolation Forest (untuned)	0.2000	6.458	—
Random Forest	0.9843	42.446	—
XGBoost	<i>0.9880</i>	<i>0.399</i>	—
LightGBM	<i>0.9886</i>	0.809	—
LSTM Autoencoder (FP32)	0.8687	0.277	0.028
Transformer Autoencoder (FP32)	0.8682	0.532	0.068
TCN-GRU (FP32)	0.9725	1.890	0.104
TCN-GRU (INT8 Quantized)	0.9723	1.935	0.048

F. Component-Wise Ablation Study

Table VIII details the impact of removing components. Because each configuration scores on a different scale, we select the threshold on the training split and apply it unchanged to test; a shared fixed threshold would measure calibration mismatch instead of component contribution. InSDN has no identity annotations, so trust comes from a simulated overlay parameterised by $\rho = P(\text{low trust} \mid \text{attack})$, reported at $\rho = 0.7$.

The result contradicts our expectation. Identity fusion and the risk engine *do not* improve $F1$; removing them nudges it from 0.9616 up to 0.9680. What they buy is false-positive suppression: FPR falls from 23.77% under direct edge scoring to 16.74%. For a deployment in which the selected action triggers autonomous isolation, trading a fraction of a point of $F1$ for fewer false alarms is operationally correct — but it is a trade, not a free gain. A sweep over $\rho \in [0.5, 0.9]$ (in the artifact) shows identity fusion improves $F1$ only once the overlay is strongly informative ($\rho \geq 0.8$).

Redis vs. REST on loopback. Transport costs are measured, not assumed: Redis Streams XADD+XREADGROUP averages 1.293 ms per event, while a synchronous localhost HTTP POST averages 0.799 ms. Consequently the “w/o Redis” arm is *faster* on this single-host harness. We retain Redis because the architecture targets an asynchronous multi-consumer cloud path, not because it wins a localhost IPC microbenchmark.

TABLE VIII

SYSTEM COMPONENT ABLATION ANALYSIS ($\rho = 0.7$, THRESHOLD TUNED ON TRAIN). LATENCIES INCLUDE MEASURED REDIS OR REST TRANSPORT; ONLY WITHIN-TABLE COMPARISON IS MEANINGFUL.

Architecture Configuration	F1-Score	FPR (%)	Latency (ms)
Full Proposed Pipeline	0.9616	16.74	4.946
— w/o Identity Context Fusion	0.9644	21.59	4.945
— w/o Cloud Risk Engine (Direct Edge Scoring)	0.9680	23.77	4.945
— w/o INT8 Quantization (FP32 Baseline)	0.9694	19.16	2.838
— w/o Redis Streams (Synchronous REST Call)	0.9616	16.74	4.452

G. Policy Control Benchmarks (RQ3)

Table IX benchmarks the cloud DQN controller against static and heuristic baselines on a single-step contention environment whose reward $r(s, a)$ trades containment efficacy against collateral QoS damage and congestion relief. Because r is analytic, the optimal action a^* is obtained by exact enumeration over the four actions, giving a ground-truth oracle. The agent is trained by fitted Q-iteration over 4,000 environment interactions with ϵ -greedy exploration and evaluated on 5,000 held-out states.

The DQN selects the oracle-optimal action in 98.8% of states and attains essentially zero mean regret, against 38.2% for a static confidence ladder and 34.8% for a hand-tuned heuristic table. Two caveats temper this. First, the agent is trained on the same reward function the oracle maximises, so this shows function-approximation capacity within a known environment, not generalisation to a live campus. Second, the DQN inflicts *more* raw QoS disruption on benign traffic (0.128) than the static ladder (0.045): reward-optimal behaviour is more aggressive than a conservative threshold rule.

TABLE IX

POLICY CONTROLLER BENCHMARK AGAINST A BRUTE-FORCE ORACLE ($n = 5,000$ HELD-OUT STATES). QoS DISRUPTION AND REGRET ARE LOWER-IS-BETTER.

Policy Mechanism	Latency (ms)	QoS Disr.	Regret	Action Acc. (%)
Static Rule Lookup	0.0008	0.045	0.080	38.2
Round-Robin Action Assignment	0.0003	0.187	0.205	25.5
Heuristic Risk Table	0.0004	0.052	0.072	34.8
Deep Q-Network (DQN Agent)	0.0551	0.128	0.000	98.8

H. How Much Did Leakage Inflate the Numbers?

Rather than assert our leakage controls were necessary, we measured what they are worth, rerunning the identical detector, seed, and schedule while changing only the split and scaler. Protocol A is the unsound one (i.i.d. split over overlapping

windows, scaler fit on the pooled corpus); Protocol B is that of Section V-B.

The effect is near-null, and this is a negative result relative to our prior expectation. On this run, Protocol A yields fixed-threshold $F1 = 0.9322$ while Protocol B yields 0.9340 ($\Delta = -0.0018$); optimal-threshold $F1$ differs by only $+0.0014$. We therefore do not claim that blocked-and-purged splitting rescued a large inflated headline number here. Two explanations fit and we cannot separate them: overlapping windows may be less informative than they look, or this corpus may be easy enough that both protocols saturate above 0.93. Blocked-and-purged splitting remains cheap and principled and we still recommend it; a reviewer should not assume a random split has by itself inflated a result by a large margin on this dataset.

TABLE X
EFFECT OF THE EVALUATION PROTOCOL. IDENTICAL MODEL, SEED, AND SCHEDULE; ONLY THE SPLIT AND SCALER DIFFER.

Protocol	$F1 @ \tau = 0.65$	τ^*	$F1 @ \tau^*$
A: random split + pooled scaler (unsound)	0.9322	0.0924	0.9693
B: blocked + purged + train-only scaler	0.9340	0.0672	0.9679
Difference (A - B)	-0.0018	—	+0.0014

I. Cross-Dataset Domain Shift Evaluation (RQ4)

Table XI evaluates zero-shot transfer from InSDN (14,000 train / 6,000 test windows) to a 20,000-window CIC-IDS2017 holdout. In-domain $F1$ is 0.9340 at $\tau = 0.65$; zero-shot $F1$ collapses to 0.4564. The failure mode is precision, not recall: the detector still recovers most attacks but its false positive rate explodes, close to a degenerate always-attack classifier. Re-tuning to $\tau^* = 0.0672$ does not rescue the transfer ($F1 = 0.4668$). One factor we did not isolate: the corpora were feature-extracted under different tooling and flow-timeout conventions, so the 10 features may not be semantically identical.

TABLE XI
CROSS-DATASET EVALUATION: TRAIN INSDN $\rightarrow$ TEST CIC-IDS2017

Protocol	Threshold τ	Precision	Recall	F1-Score
InSDN $\rightarrow$ InSDN	0.65	0.9777	0.8941	0.9340
InSDN $\rightarrow$ InSDN	$\tau^* = 0.0672$	0.9461	0.9906	0.9679
InSDN $\rightarrow$ CIC-IDS2017	0.65	0.3059	0.8980	0.4564
InSDN $\rightarrow$ CIC-IDS2017	$\tau^* = 0.0672$	0.3052	0.9913	0.4668

VII. THREATS TO VALIDITY & LIMITATIONS

A. Threats to Validity

- **Internal Validity (measurement stability):** Timing on this host is noisy enough that single measurements are not results. Every timing figure is now a median over repeats — 5 for throughput and per-model latency, 1,000 trials summarised by percentiles for the control path — after we found that a single-run measurement had produced

a throughput shortfall that repetition did not reproduce. Saturation throughput remains unstable between runs by more than a factor of two and we rest no claim on it. Accuracy metrics, by contrast, were bit-identical across all runs. Absolute latencies should be read as order-of-magnitude on this hardware, not as hardware-independent guarantees.

- **Reproducibility:** An earlier version of this artifact constructed models before seeding the global RNG, so detection metrics varied between runs of the same script. `set_global_seed` now runs at the head of every generator, before any module is built, making each script bit-deterministic. It is *not* re-invoked before each individual model, so a model built after several others inherits an RNG advanced by their construction — which is exactly why the same detector scores 0.9679 in Section VI-B and 0.9723 in Section VI-E. Determinism at a fixed seed is not robustness across seeds; accuracy is single-seed throughout.
- **External Validity:** Evaluation uses public InSDN and CIC benchmarks; broader IoT/IIoT corpora such as UNSW-NB15 [6], ToN_IoT [7], and Edge-IIoTset [8] offer complementary vectors for multi-campus validation.
- **Construct Validity:** Metrics assume clean flow labels; noisy headers could affect precision. We report no human-SOC comparison: we have no measured analyst baseline, and a stipulated one would establish only that milliseconds beat minutes, which needs no experiment.
- **Residual Leakage Risk:** Purging removes shared timestep *indices* over the post-shuffle window order, bounding stride-1 index-overlap leakage rather than wall-clock or session isolation. Session-aware grouping would be stronger; InSDN lacks the metadata, and our 50k subsample was shuffled before windowing.

B. Limitations & Future Work

Primary limitations: (i) no executable Mininet/Ryu dataplane in this artifact and no measured `flow_mod` install; (ii) no evaluation of adversarial robustness against evasion; (iii) severe zero-shot cross-dataset degradation; (iv) rejection of H2 on its CPU term, costing a median 36.0% of one logical CPU against a 10% budget at 10,000 windows/s; (v) a detector outperformed on these features by off-the-shelf gradient boosting, leaving its case resting on footprint alone; (vi) Redis Streams measured on localhost, so cross-host network RTT is uncoupled, and localhost REST can beat Redis on IPC alone; (vii) single-seed reporting of accuracy; (viii) identity context remains a simulated overlay; and (ix) the 4.793 ms control-path median is a batch-1/sequential path, whereas the 10,000 windows/s throughput figure uses `batch = 128`, so the two results do *not* jointly prove sustained sub-10 ms decisions at that offered load once batching and queueing delay are counted. Future work: dataplane integration, domain adaptation, compiled ONNX/TensorRT edge runtimes, online threshold tuning, multi-seed reporting, and federated multi-campus learning.

VIII. ETHICS, PRIVACY & PUBLIC ARTIFACT

A. Privacy Posture

The architecture operates on non-DPI flow headers. Edge identity lookup uses a simulated lease table; cloud-bound Redis payloads carry only HMAC-SHA256 principal_id values (keyed by ICAI_PSEUDONYM_SECRET or a documented development key), not cleartext IP/MAC. Unkeyed hashing is intentionally avoided. DPI is omitted. This supports data-minimization goals under GDPR and Vietnam's Decree 13/2023 without claiming production anonymity.

B. Public Research Artifact

Code, frozen window archives, window-rebuild recipe, checkpoints, Docker files, and scripts are available under MIT License at <https://github.com/thtsec/icaifai2026>. Every quantitative result table in Section VI is regenerated by `results/run_all_and_summarize.py` (seed 42), which emits `results/paper_metrics_summary.json`. Dataset SHA-256 manifests live under `datasets/`.

IX. CONCLUSION

This paper presented an AI-native event-driven edge-cloud architecture for autonomous campus network security. Combining INT8-quantized TCN-GRU edge detectors, HMAC-pseudonymized Redis Streams transport, and cloud DQN action selection and orchestration, it achieves a software control path with a 4.793 ms median and a 7.202 ms p99 on an Intel Core i7-13620H host with localhost Redis 7.2.4, from prepared-window inference to playbook construction. We stop short of calling this closed-loop mitigation: dataplane enforcement is unmeasured, and we report no measured human-SOC comparison.

The results we think transfer beyond this system are the negative ones. Gradient-boosted trees on the same features beat the proposed detector ($F1 = 0.9886$ for LightGBM against 0.9723). Reconstruction-error scoring is unusable on an attack-heavy corpus ($F1 \approx 0.035$). Leakage from stride-1 overlap is near-null on this run ($|\Delta F1| < 0.002$). H2 fails on its CPU term, sustaining 10,000 windows/s but at 36.0% of one logical CPU against a 10% budget. Zero-shot transfer to CIC-IDS2017 collapses to $F1 = 0.456$. Together these bound where an AI-native campus defense is currently practical, and they point the architecture's value at integration and orchestration rather than at the choice of detector.

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology at HUFLIT for academic and computational support.

REFERENCES

- [1] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurec, "InSDN: A Novel SDN Intrusion Dataset," *IEEE Access*, vol. 8, pp. 165263–165284, 2020, doi: 10.1109/ACCESS.2020.3022633.
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018.
- [4] S. Kianpisheh and T. Taleb, "Collaborative Federated Learning for 6G With a Deep Reinforcement Learning-Based Controlling Mechanism: A DDoS Attack Detection Scenario," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, Aug. 2024, doi: 10.1109/TNSM.2024.3387987.
- [5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1–6, doi: 10.1109/MILCIS.2015.7348942.
- [7] A. Alsaedi *et al.*, "TON-IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Cybersecurity Applications," *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [8] M. A. Ferrag *et al.*, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IIoT and IIoT Applications for Centralized and Federated Learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022, doi: 10.1109/ACCESS.2022.3165809.
- [9] N. McKeown *et al.*, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [10] Ryu SDN Framework Community, "Ryu SDN Framework," 2020. [Online]. Available: <https://ryu-sdn.org/>
- [11] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.
- [12] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 2019, doi: 10.1109/COMST.2019.2904897.
- [13] T. A. Tang *et al.*, "Deep recurrent neural network for intrusion detection in SDN-based networks," in *Proc. 4th IEEE Conf. Netw. Softwarization (NetSoft)*, 2018, pp. 202–206, doi: 10.1109/NETSOFT.2018.8460090.
- [14] M. Roopak, G. Yun Tian, and J. Chambers, "Deep learning models for cyber security in IoT networks," in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, 2019, pp. 0452–0457, doi: 10.1109/CCWC.2019.8666588.
- [15] P. H. Nguyen *et al.*, "Towards Smarter Security Orchestration and Automatic Response for CPS and IIoT," in *Proc. 14th IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, 2023, pp. 298–302, doi: 10.1109/CloudCom59040.2023.00055.
- [16] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010, pp. 1–6, doi: 10.1145/1868447.1868466.
- [17] M. B. Musthafa *et al.*, "Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices," *IEEE Access*, vol. 13, pp. 113544–113556, 2025, doi: 10.1109/ACCESS.2025.3584373.
- [18] S. Ullah *et al.*, "Quantized Intrusion Detection for Resource-Constrained IIoT: A Comparative Evaluation of Efficiency and Adversarial Robustness," *Comput. Mater. Contin.*, vol. 88, no. 3, 2026, doi: 10.32604/cmc.2026.084409.
- [19] S. Srinivas *et al.*, "AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation," *J. Cybersec. Privacy*, vol. 5, no. 4, art. 95, 2025, doi: 10.3390/jcp5040095.
- [20] D. Arp *et al.*, "Dos and Don'ts of Machine Learning in Computer Security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.
- [21] M. A. Bouke and A. Abdullah, "An empirical assessment of ML models for 5G network intrusion detection: A data leakage-free approach," *e-Prime*, vol. 8, art. 100590, 2024, doi: 10.1016/j.prime.2024.100590.
- [22] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proc. NeurIPS*, 2017, pp. 3146–3154.
- [23] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [24] Y. Liu, M. Dong, K. Ota, J. Li, and J. Wu, "Deep Reinforcement Learning based Smart Mitigation of DDoS Flooding in Software-Defined Networks," in *Proc. IEEE CAMAD*, Barcelona, Spain, 2018, pp. 1–6, doi: 10.1109/CAMAD.2018.8514971.
- [25] N. M. Yungaicela-Naula, C. Vargas-Rosales, J. A. Pérez-Díaz, and M. Zareei, "Towards security automation in Software Defined Networks," *Comput. Commun.*, vol. 183, pp. 64–82, Feb. 2022, doi: 10.1016/j.comcom.2021.11.014.